



Module Title

Cybersecurity and Networking

Internship Assignment Submission

Software Supply Chain Security

Student Name: Jiwosh Shrestha

Internship ID: JVL-CS-2026-8990

Role: Cybersecurity Intern

Duration: 3 months

Assignment Submission Date: Sunday, 4th October 2026

Day:25 (Twenty-Five)

Table of Contents

1. DAY 25 ASSIGNMENT A03:2025 SOFTWARE SUPPLY CHAIN FAILURES	3
1.1 Answer all questions clearly in your own words.	3
1.1.1 PART A — FUNDAMENTALS	3
1.1.3 PART B — SBOM & VULNERABILITY MANAGEMENT	5
1.1.3 PART C — CI/CD SECURITY	7
1.1.4 PART D — PRACTICAL ANALYSIS	8
1.1.5 PART E — CI/CD CASE.....	10

1. ASSIGNMENT A03:2025 SOFTWARE SUPPLY CHAIN FAILURES

1.1 Answer all questions clearly in your own words.

1.1.1 PART A — FUNDAMENTALS

1. What is a software supply chain?

A software supply chain is everything involved in creating, building, testing, delivering, and updating software. It includes code, dependencies, tools, people, and process.

2. What is a software dependency?

A software dependency is a library, package, framework, or other component that an application needs to work properly.

3. Explain a direct dependency.

A direct dependency is a component that developers intentionally add and use in their application. For example, an application directly using a Python library.

4. Explain a transitive dependency.

A transitive dependency is a dependency required by another dependency. For example, if Application uses Library A and Library A uses Library B, Library B is a transitive dependency.

5. Why can outdated components create security risk?

Outdated components may contain known security vulnerabilities and may not have the latest security fixes. They can make an application easier to attack.

6. Why can unmaintained components be a concern?

Unmaintained components may no longer receive security updates or bug fixes. This can leave known problems unresolved.

7. Why should software come from trusted sources?

Trusted sources help ensure that software is genuine and has not been replaced or modified by an unknown party. It also makes the software's origin easier to verify.

8. Why should unused dependencies be removed?

Unused dependencies add unnecessary complexity and potential security risks. Removing them can reduce the application's attack surface.

1.1.3 PART B — SBOM & VULNERABILITY MANAGEMENT

9. What does SBOM stand for?

SBOM stands for Software Bill of Materials.

10. Explain an SBOM in your own words.

An SBOM is like an ingredients list for software. It shows the components and dependencies used to build an application.

11. Give three types of information an SBOM can help an organization understand.

An SBOM can help identify:

Software components used

Component versions

Dependency relationships

12. Does an SBOM automatically fix vulnerabilities? Explain.

No. An SBOM only provides visibility into the software components. Security teams still need to identify, assess, and fix vulnerabilities.

13. What is a CVE?

CVE stands for Common Vulnerabilities and Exposures. It is standard identifier used to identify publicly known security vulnerabilities.

14. Why should organizations monitor security advisories?

Security advisories provide information about newly discovered vulnerabilities and available fixes. Monitoring them helps organizations respond to security issues in the software they use.

15. What is patch management?

Patch management is the process of identifying, testing, and applying software updates or security fixes to systems and applications.

16. Why should updates be tested before production deployment?

Testing helps identify compatibility problems, errors, or unexpected behavior before the update affects real users and production systems.

1.1.3 PART C — CI/CD SECURITY

17. What is CI/CD at a beginner level?

CI/CD is a system used to automatically build, test, package and sometimes deploy software. It helps developers deliver software more efficiently.

18. Why is CI/CD part of the software supply chain?

CI/CD is involved in building, testing, packaging, and deploying software. Therefore, a security problem in the pipeline can affect the final application.

19. Explain Least Privilege for a CI/CD system.

Least Privilege means giving CI/CD accounts and jobs only the permissions they need to perform their tasks. Unnecessary administrative access should be avoided.

20. Why should production secrets not be stored openly in source-code repositories?

Secrets such as password, API keys, and access tokens could be exposed to unauthorized users. They should be stored using secure secret-management methods.

1.1.4 PART D — PRACTICAL ANALYSIS

Answer:

1. Which component is unmaintained?

Legacy-report-parser

2. Which unused component should be considered for removal?

Old-ui-helper

3. Which component contains an unsupported transitive dependency?

Auth-library

4. Name that transitive dependency.

Legacy-token-helper

5. Which component came from an unknown source?

Random-file-tool

6. Why is that a security concern?

The source and trustworthiness of the component cannot be confidently verified. It may contain unsafe or modified code.

7. How could an SBOM help?

An SBOM could provide visibility into the applications components, versions, and dependency relationships, making it easier to identify risks.

1.1.5 PART E — CI/CD CASE

Answer:

1. Identify the positive security control.

Repository MFA: Enabled

MFA provides an additional layer of protection for repository accounts and helps reduce the risk of unauthorized access.

2. Identify the four main security concerns.

I. Build Server Admin Access: All Developers – Developers have more privileges than they may need.

II. Production Secret – Sensitive credentials could be exposed.

III. Dependency Inventory: Not Maintained – The organizations may not know which components and versions are being used.

IV. Updates: Deployed Directly to Production Without Testing – Untested updates can cause security, compatibility, or operational problems.

3. Give four defensive recommendations.

I. Apply Least Privilege and limit build-server administrative access.

II. Remove production secrets from the repository and use secure secret management.

III. Maintain an updated dependency inventory and SBOM where appropriate.

IV. Test updates before production deployment and use a controlled or staged rollout.